Jose Pablo Canton Monzón
26009
Programación Orientada a Objetos

Laboratorio #2

Análisis

Requisitos Funcionales

ID	Módulo	Descripción Funcional
RF-01	Gestión del Caso	El sistema debe solicitar e inicializar los datos generales del caso: Nombre, Código de identificación y Detective responsable.
RF-02	Ubicaciones (Fijas)	Administrar un arreglo básico de tamaño fijo igual a 5 celdas para almacenar objetos de tipo Ubicacion.
RF-03	Registro de Ubicación	Permitir registrar una ubicación indicando la posición exacta (0 a 4), validando disponibilidad previa (comprobación de null).
RF-04	Consulta & Modificación	Permitir listar ubicaciones activas, consultar una específica por índice, y modificar únicamente riesgo (1-10) y estado.
RF-05	Descarte de Ubicación	Permitir liberar una celda ocupada del arreglo, reasignando explícitamente el valor null en dicho índice.
RF-06	Pistas (Dinámicas)	Almacenar un número indeterminado de evidencias mediante una colección dinámica ArrayList<Pista>.
RF-07	Operaciones de Pistas	Soportar el registro (sin códigos duplicados), búsqueda secuencial por código, modificación y eliminación del ArrayList.

ID	Módulo	Descripción Funcional
RF-08	Reporte de Investigación	Calcular métricas: conteo de ubicaciones/pistas, espacios libres, ubicación con mayor riesgo, pistas con mayor importancia y confiabilidad, y promedio de importancia.
RF-09	Manejo de Excepciones	Controlar InputMismatchException (entradas no numéricas) e IllegalArgumentException (rangos inválidos) sin estrellar el sistema.

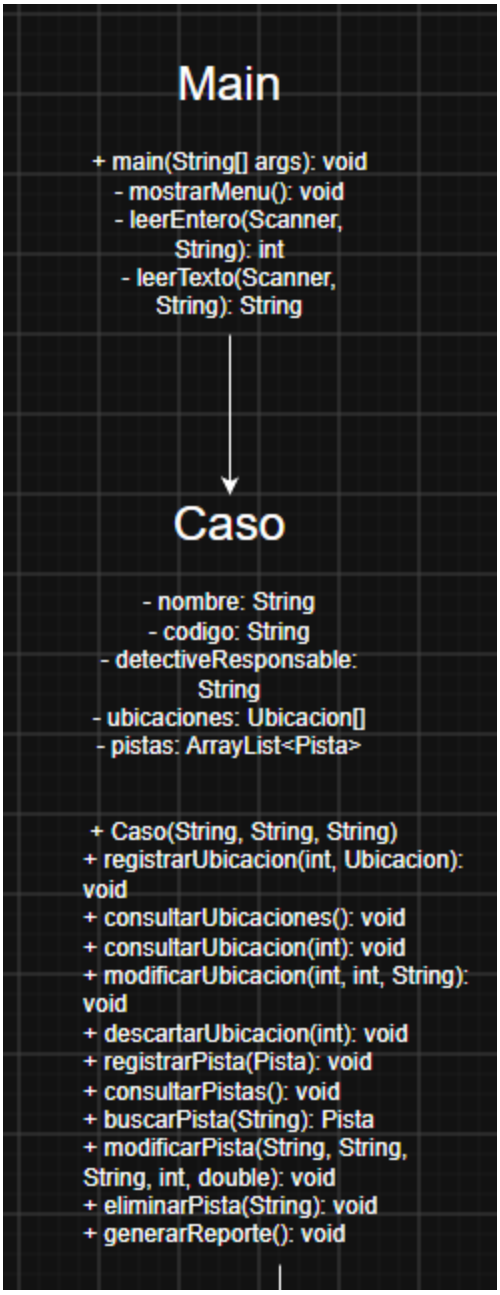
Propósitos de Clase

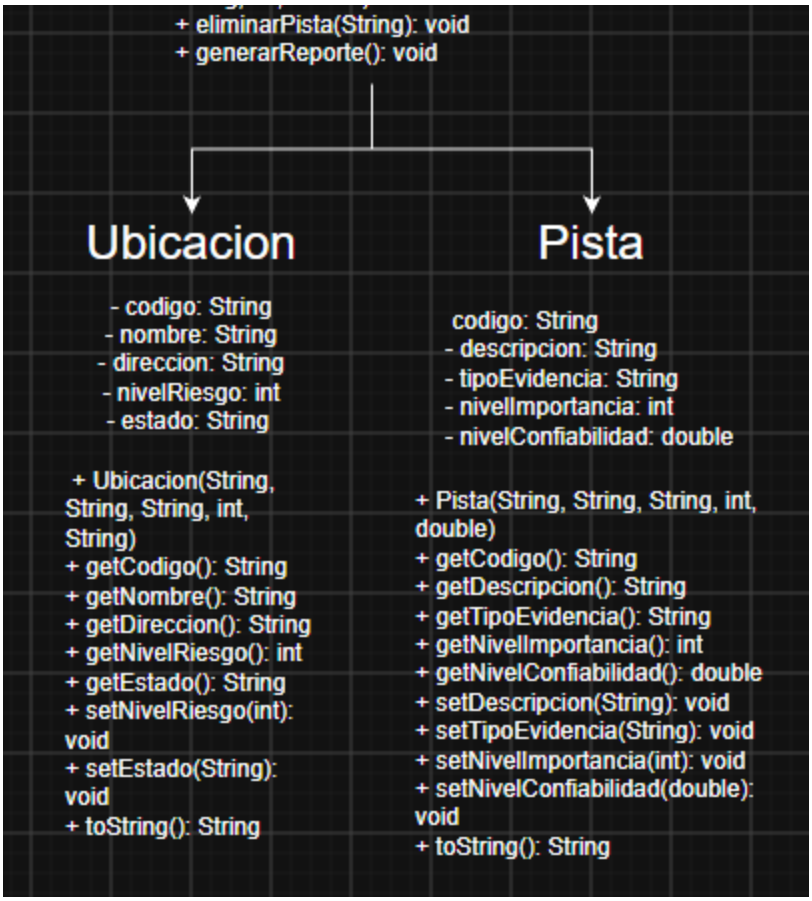
1. Ubicación
    a. Representa un punto geográfico o sitio de interés de la investigación. Encapsula código, nombre, dirección, nivel de riesgo (1-10) y estado operativo. Valida sus invariantes de rango.
2. Pista
    a. Modela un elemento de evidencia hallado. Encapsula código único, descripción, tipo de evidencia, nivel de importancia (1-10) y nivel de confiabilidad (0-100).
3. Caso
    a. Clase principal de dominio. Administra la estructura fija Ubicacion[5] y la lista dinámica ArrayList<Pista>. Contiene los algoritmos de búsqueda, cálculo de reportes y validaciones.
4. Main
    a. Clase con el método main(). Despliega la interfaz de usuario por consola, gestiona la captura con Scanner, y contiene los bloques try-catch-finally.

Propiedades y Métodos de clase

Clase	Atributos y Tipos	Métodos Principales (Firmas)
Ubicacion	- codigo: String - nombre: String - direccion: String - nivelRiesgo: int - estado: String	+ Ubicacion(...) + getCodigo(): String + getNivelRiesgo(): int + setNivelRiesgo(n: int): void + setEstado(e: String): void
Pista	- codigo: String - descripcion: String - tipoEvidencia: String - nivelImportancia: int - nivelConfiabilidad: int	+ Pista(...) + getCodigo(): String + getNivelImportancia(): int + getNivelConfiabilidad(): int + setNivelImportancia(i: int): void
Caso	- nombreCaso: String - codigo: String - detectiveResponsable: String - ubicaciones: Ubicacion[] - pistas: ArrayList<Pista>	+ Caso(nombre: String, cod: String, det: String) + registrarUbicacion(pos: int, u: Ubicacion): boolean + registrarPista(p: Pista): boolean + buscarPista(cod: String): Pista + calcularPromedioImportancia(): double

UML


Main
+ main(String[] args): void
- mostrarMenu(): void
- leerEntero(Scanner, String): int
- leerTexto(Scanner, String): String
Caso
- nombre: String
- codigo: String
- detectiveResponsable: String
- ubicaciones: Ubicacion[]
- pistas: ArrayList<Pista>
+ Caso(String, String, String)
+ registrarUbicacion(int, Ubicacion): void
+ consultarUbicaciones(): void
+ consultarUbicacion(int): void
+ modificarUbicacion(int, int, String): void
+ descartarUbicacion(int): void
+ registrarPista(Pista): void
+ consultarPistas(): void
+ buscarPista(String): Pista
+ modificarPista(String, String, String, int, double): void
+ eliminarPista(String): void
+ generarReporte(): void


+ eliminarPista(String): void
+ generarReporte(): void
Ubicacion
- codigo: String
- nombre: String
- direccion: String
- nivelRiesgo: int
- estado: String
+ Ubicacion(String, String, String, int, String)
+ getCodigo(): String
+ getNombre(): String
+ getDireccion(): String
+ getNivelRiesgo(): int
+ getEstado(): String
+ setNivelRiesgo(int): void
+ setEstado(String): void
+ toString(): String
Pista
codigo: String
- descripcion: String
- tipoEvidencia: String
- nivelImportancia: int
- nivelConfiabilidad: double
+ Pista(String, String, String, int, double)
+ getCodigo(): String
+ getDescripcion(): String
+ getTipoEvidencia(): String
+ getNivelImportancia(): int
+ getNivelConfiabilidad(): double
+ setDescripcion(String): void
+ setTipoEvidencia(String): void
+ setNivelImportancia(int): void
+ setNivelConfiabilidad(double): void
+ toString(): String